

DEAKIN UNIVERSITY

SOFTWARE ARCHITECTURE AND SCALABILITY FOR INTERNET-OF-THINGS

ONTRACK SUBMISSION

Module 1 Learning Evidence

Submitted By:

Jahan GARG

s226100103

2026/07/26 23:59

Tutor:

Nisha KUMARI

Task Outcomes		Supports
TLO1	Demonstrate engagement with the following unit learning outcomes (legacy)	ULO1 Demonstrate ULO2 Apply ULO3 Develop ULO4 Deploy

July 26, 2026



Week 1, 2026 Thursday 9th July

Week 1, 2026 Thursday 9th July

View only

TJ

SIT314 - Week 1 Seminar

How well you understand?

TJ

Waiting

TJ

Just Another Classroom A

Class Discussion

What is a Learning?

Why does Technology Use

Why does it need to be used?

SIT314 - Week 1 Seminar

10:41

06-07-2026

Week 1, 2026 Thursday 9th July

Week 1, 2026 Thursday 9th July

View only

TJ

SIT314 - Week 1 Seminar

How well you understand?

TJ

Waiting

TJ

Just Another Classroom A

Class Discussion

What is a Learning?

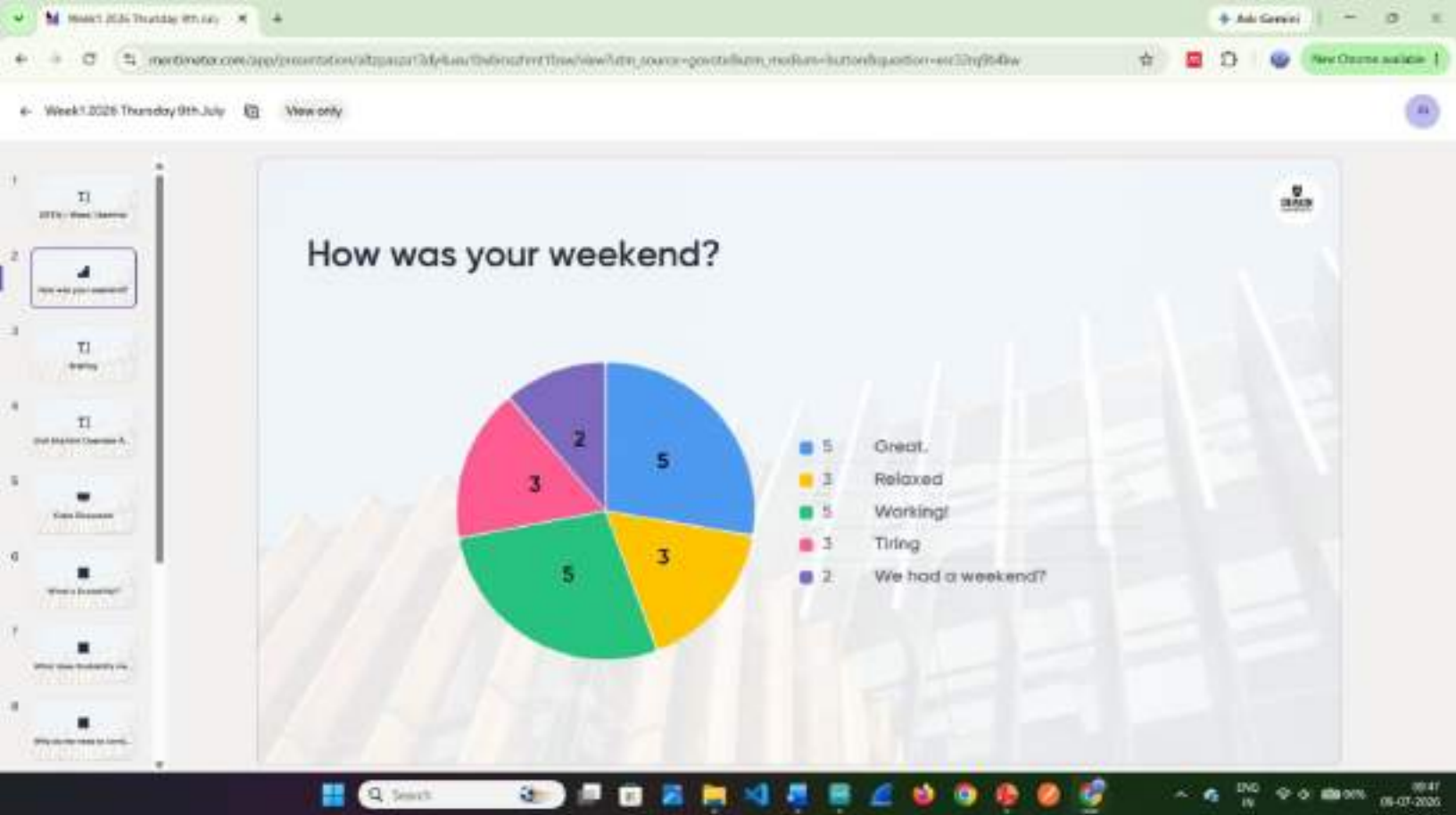
Why does Technology Use

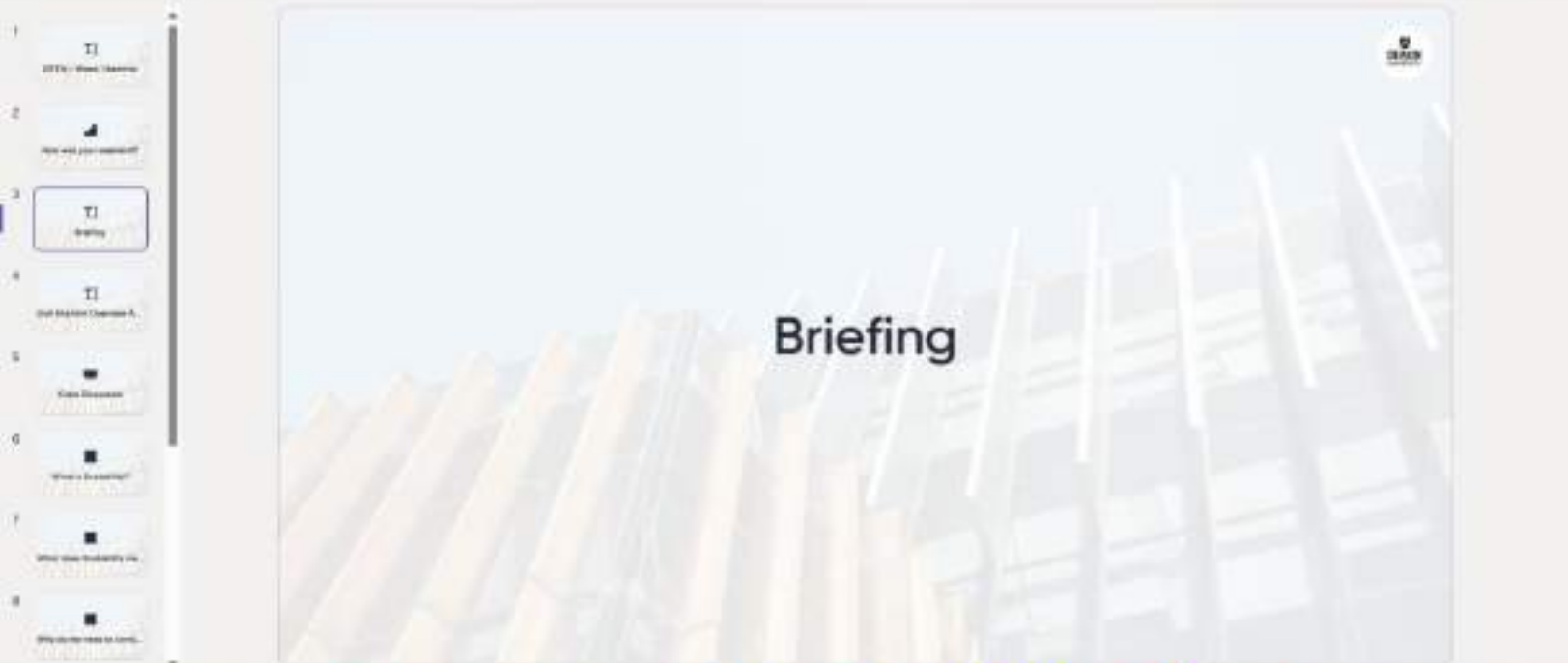
Why does it need to be used?

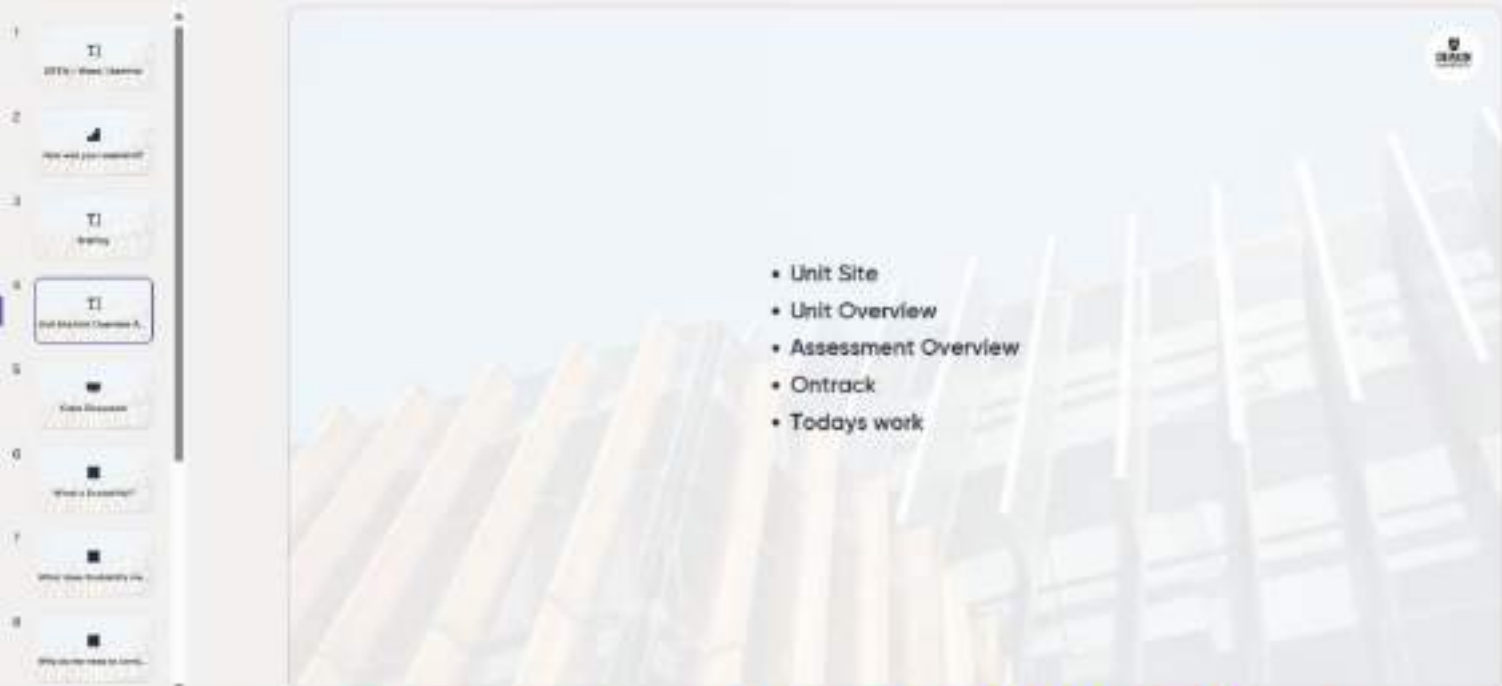
SIT314 - Week 1 Seminar

10:41

06-07-2026









Class Discussion

Week1 2026 Thursday 9th July

Week1 2026 Thursday 9th July

View only

meritnode.com/app/presentation/altipassr/3dylau/0b0c0f717a/view?utm_source=google&utm_medium=button&position=1&aggressive=1

New Course available

1

2026 - Week 1 Overview

2

How will you measure it?

3

TI

Scaling

4

TI

Just Another Challenge A

5

View Overview

6

What is Scalability?

7

Why does Scalability fail?

8

Why does it need to work?

What is Scalability?

scale-up and scale-down

expanding effectively

upgrade and expand

the way to scale up

able to grow infrastructure

how it handle work

capacity to handle growth

ability to grow

expansion

expanding efficiently

ability to expand

expand to growth

size and amount

able to expend

adjustable threshold

expand the range

expanding the software

system load to handle user

improving and upgrading

10:47

06-07-2026

Week 1, 2026 Thursday 9th July

View only

1

IoT - Week 1 Overview

2

How will you connect it?

3

IoT - Scaling

4

IoT - Scaling Challenges A

5

Cloud Overview

6

Where is Scalability?

7

What does Scalability mean?

8

Why does it need to scale?

What does Scalability mean in computing and IoT?

increase work capacity

handle different devices

increase in efficiency

elasticity

high performance

handling large datasets

efficiency and functional

increase of devices

expand efficiently

ability to grow

handle more without loss

handling I data efficient

efficiency increase

handle more devices

expandable and upgradable

handle growing workloads

performance n efficiency

performance

10:47

06-07-2026

Week 1, 2025 Thursday 9th July

View only

TI

What is a smart home?

TI

How can you connect?

TI

Scaling

TI

What is the cloud?

TI

Cloud Storage

TI

What is a smart city?

TI

What is a smart city?

TI

Why do we need to scale?

Why do we need to consider Scalability in IoT?

support more devices

handles future growth

productivity

future growth

no compromised systems

we can't add more systems

load on the hardware

load and data handling

so it doesn't stop or crash

for smooth working

delays in crashes

can't handle large users

downtime

fault tolerance

purpose- fault tolerance

hardware capability

functionality

versatile the connection

10:47

08-07-2025

SIT314/SIT729 – Week 1 Group Activity

The Impact of IoT on our lives

Overview

In this task, you are going to think about the impact scalability has on the IoT services that we use.

Group: Ayush, Jahan, Andres

Tasks

1. What Internet-of-Things devices, applications, What services do you use at home?

- Google Home
- Security cameras
- Smart TV
- Smart bulbs / LED lights
- Video doorbell
- Smart refrigerator

These devices rely on home Wi-Fi and cloud services for automation, remote control, and data collection.

2. What Internet-of-Things devices, applications, or services do you use at university, at work, when traveling, or when outside your home?

- Student ID access card
- Wi-Fi-connected printer
- Smart projector
- Smart watch
- Traffic systems

These IoT systems support secure access, printing, teaching, personal health tracking, and real-time traffic management.

3. Quantify the amount of storage/data transmission/processing required for the following IoT applications.

For each, list the type of information that it collects. Examples include key presses, favorite choices, location, and distance traveled. Calculate the storage required for that application for a day/month/year.

Andres

Health Monitoring using a smart watch, heart rate monitor, and internet connected weight scales.

A smart watch collects a lot of data and information such as, steps, heart rate, training status, sleep, energy levels, VO2, calories burned, SpO2, stress, location, app usage, height, weight, gender, age and elevation.

The storage required for this can vary, as many watches have varying features but a rough estimate would be:

- 2-5 MB a day
- 75-150 MB a month
- 800MB-1.8 GB a year

These estimates are on the right track because the information that is collected is mainly just numbers and text, where the files are pretty small. With about 3-6 GB of storage in modern day smart watches.

Jahan

Fleet management using IoT devices to track and manage vehicles in logistics and transportation

IoT-based fleet management uses GPS trackers and vehicle sensors to monitor each vehicle in real time. Typical data includes location (coordinates, speed, route), fuel usage, engine status, mileage, and driver behavior events such as harsh braking or overspeeding. [1] [2]

A modern telematics unit can generate roughly 1–3 GB of data per vehicle per day when streaming frequent GPS updates and sensor readings to the cloud. That scales to about 30–90 GB per month and 360–1080 GB (0.36–1.08 TB) per year per vehicle, so large fleets must plan storage, bandwidth, and retention policies carefully to keep the IoT system scalable. [3] [4]

References:

[1]

<https://www.korewireless.com/blog/what-automotive-telematics-actually-demands-from-a-fleet-iot-network/>

[2]

<https://trafalgairwireless.com/blog/the-complete-guide-to-iot-in-fleet-management/>

[3]

<https://docs.aws.amazon.com/whitepapers/latest/designing-next-generation-vehicle-communication-aws-iot/intelligent-data-collection-with-aws-iot-fleetwise.html>

[4]

<https://uraeus.io/resources/scaling-vehicle-telemetry/>

Ayush

Smart Agriculture using IoT devices for **precision farming, crop monitoring, and livestock management**. Smart agriculture collects data such as **soil moisture, temperature, humidity, rainfall, crop health, water usage, livestock location, and animal health information**.

The storage required can vary depending on the number of sensors used, but a rough estimate would be: **100–300 MB a day or 3–9 GB a month or 36–108 GB a year**

These estimates are reasonable because sensors collect information throughout the day and send it to the cloud for analysis, helping farmers monitor crops and livestock more efficiently.

References:

IBM. (n.d.). *What is smart agriculture?*

<https://www.ibm.com/think/topics/smart-agriculture>

Wolfert, S., Ge, L., Verdouw, C., & Bogaardt, M. J. (2017). *Big data in smart farming – A review*. *Agricultural Systems*, 153, 69–80.

<https://doi.org/10.1016/j.agsy.2017.01.023>

4. Imagine a futuristic home in which everything is fully automated with cloud-based IoT automations.

What would be the consequences if the internet service were cut off and there were no local automation services available?

Ayush

The home air conditioning system

If the internet service was cut off and there were no local automation services, the smart air conditioning system would stop working automatically. Users would not be able to control it through their **phone or voice assistant**, and scheduled temperature settings would not work. It will make it tough to survive the extreme hot and cold.

Andres

The Security Access and Alarm System

There would be many consequences to a fully automated futuristic home where the internet service is cut off. With a security access and alarm system, the cameras and locks would stop working. This poses a serious threat as cameras are preventative but locks are active; if the locks of a home do not work then anyone with bad intentions can walk in. These consequences are serious; as if there was an intruder, the alarm system would be useless, posing a threat to anyone in the home, as there would not be a formal alarm going off. Without a local automation system, these functions are useless and might as well not be there, showing us that there must be a local automation service that exists or we do not rely so much on fully automated systems.

Jahan

Smart fire, gas, and carbon monoxide monitoring system

In a fully automated home where fire, gas, and CO monitoring relies entirely on cloud-based IoT, the system depends on continuous internet connectivity for detection and response. If the internet is cut and there is no local automation, sensor data cannot be analyzed in real time, and no alarms, push notifications, or automatic safety actions (such as shutting off gas or turning off HVAC) will be

triggered. [1] [2]

This creates a dangerous “silent failure”: the home appears smart and protected, but during an outage, it effectively behaves like it has no working safety system. Research on smart fire and gas systems, therefore, recommends combining cloud features with local processing, so critical alarms and shutoff actions still work even when the network goes down. [3] [4]

References:

[1]

https://thesai.org/Downloads/Volume13No5/Paper_6-The_Design_of_Home_Fire

[2]

https://www.metrology-journal.org/articles/ijmqe/full_html/2025/01/ijmqe240032/ijmqe240032.html

[3]

<https://ijrpr.com/uploads/V7ISSUE5/IJRPR64329.pdf>

[4]

https://thesai.org/Downloads/Volume13No10/Paper_99_An_Integrated_Hardware_Prototype_for_Monitoring_Gas_Leaks_Fires.pdf

SIT314 Week-1 Technical Task Report – TCP Socket Client–Server Experiments

Overview

This report documents the design, implementation, and experimental evaluation of a simple TCP socket client–server application developed for SIT314 Week-1. The goal was to start from the provided baseline code and systematically modify the client and server to increase load, explore failure modes, and understand how TCP behaves under stress.

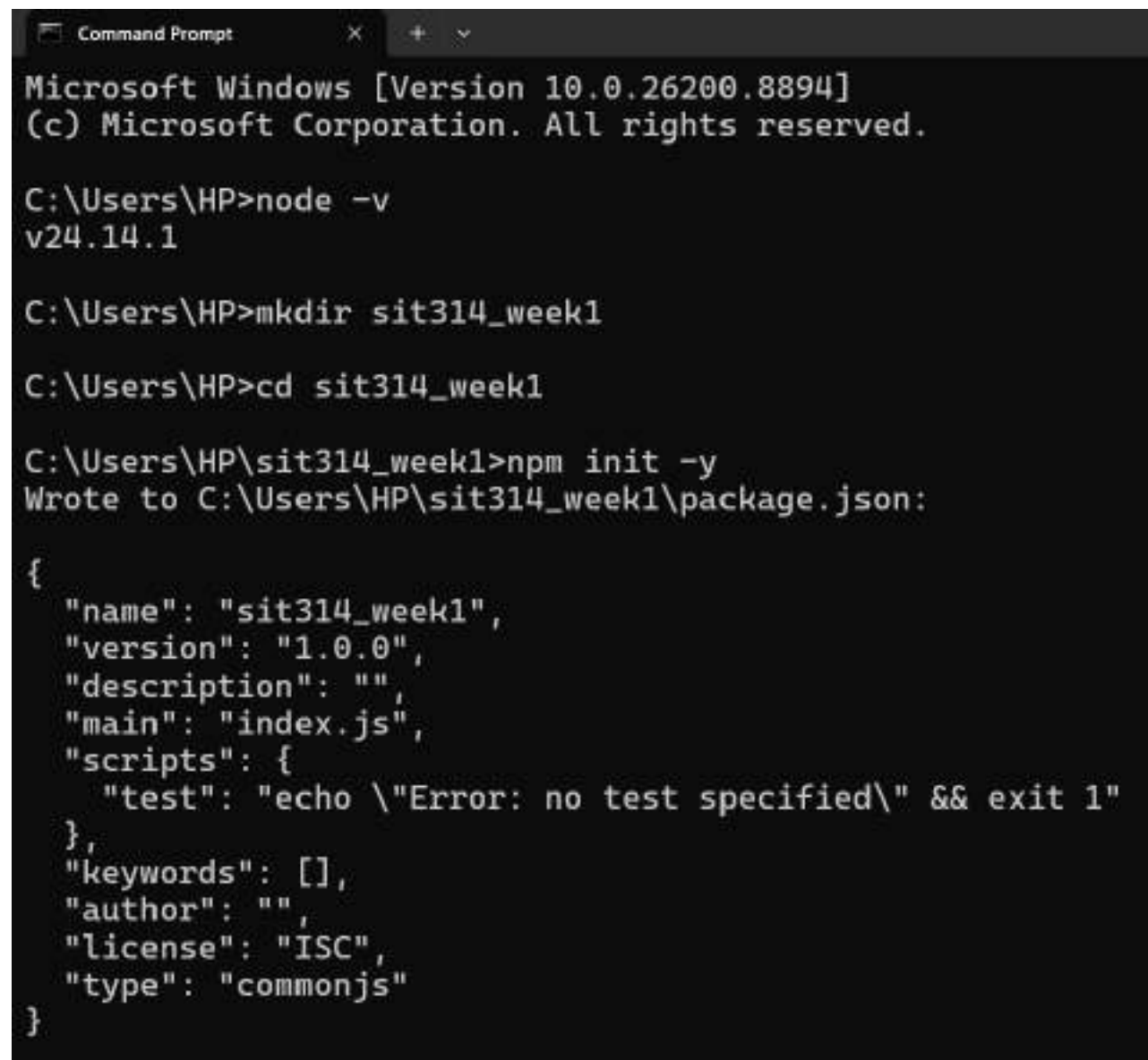
The application is implemented in Node.js using the built-in `net` module, with a server that performs basic arithmetic on numeric operands and a client that generates and sends calculation requests at configurable intervals.[1]

1. Project Setup and Baseline Behaviour

1.1 Initial Folder and Files

The project was created in the SIT314_WEEK1 folder and initialised with the following files:

- package.json – Node.js metadata for the project.
- socket_server.js – TCP server implementation.
- socket_client.js – TCP client implementation.

A screenshot of a Windows Command Prompt window. The title bar says 'Command Prompt'. The text inside shows the following commands and their output:
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>node -v
v24.14.1

C:\Users\HP>mkdir sit314_week1

C:\Users\HP>cd sit314_week1

C:\Users\HP\sit314_week1>npm init -y
Wrote to C:\Users\HP\sit314_week1\package.json:

{
 "name": "sit314_week1",
 "version": "1.0.0",
 "description": "",
 "main": "index.js",
 "scripts": {
 "test": "echo \"Error: no test specified\" && exit 1"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "type": "commonjs"
}

Screenshot 1 – Project folder structure and initial files (SIT314_WEEK1 with package.json, socket_server.js, socket_client.js).

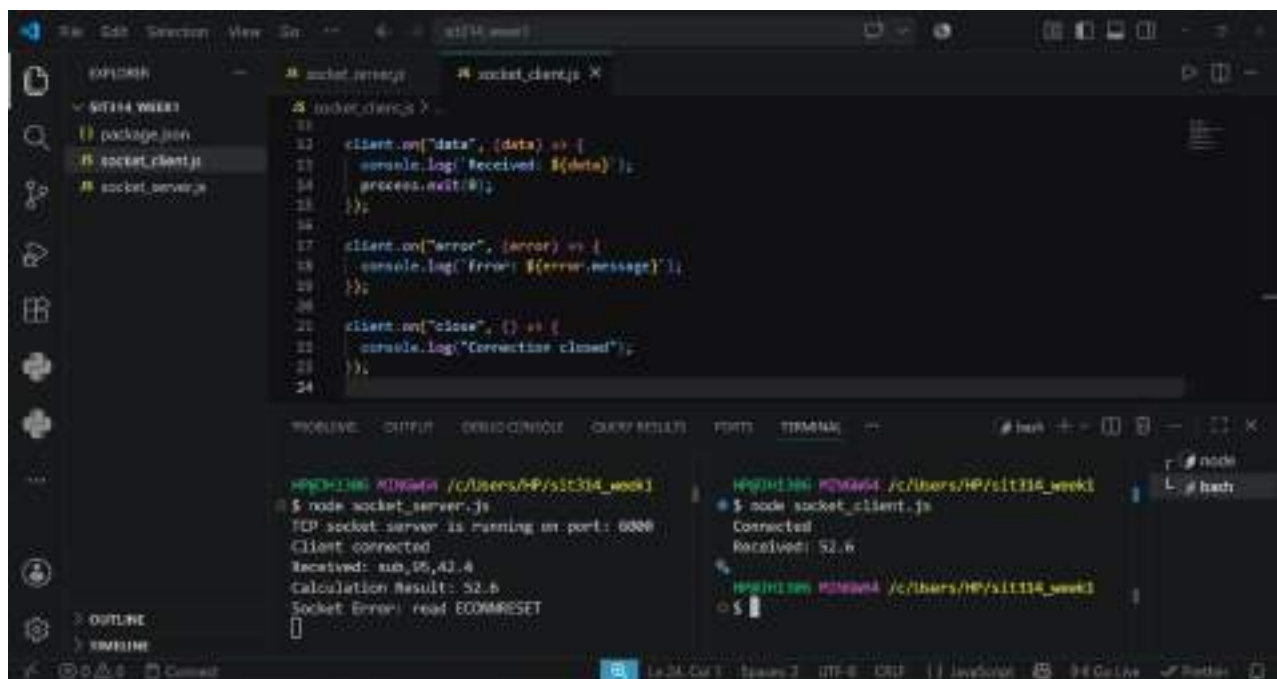
This screenshot confirms that the technical task was started from the required structure and that both client and server files were present before experimentation.

1.2 Single Client-Server Exchange (Baseline)

The baseline test used the original, mostly unmodified code provided in the task, with a single client instance connecting to the server and sending a small number of requests at the default interval.[1]

The server behaviour for each request was:

- Accept a TCP connection.
- Read a comma-separated string operator,num1,num2 from the socket.
- Parse the operator and numeric operands.
- Perform the requested arithmetic operation.
- Send the numeric result back to the client.



```
socket_server.js
11 client.on("data", (data) => {
12   console.log('Received: ' + (data));
13   process.exit(0);
14 });
15
16 client.on("error", (error) => {
17   console.log('Error: ' + (error.message));
18 });
19
20 client.on("close", () => {
21   console.log('Connection closed');
22 });
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
socket_client.js
11 client.on("data", (data) => {
12   console.log('Received: ' + (data));
13   process.exit(0);
14 });
15
16 client.on("error", (error) => {
17   console.log('Error: ' + (error.message));
18 });
19
20 client.on("close", () => {
21   console.log('Connection closed');
22 });
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
Terminal 1
$ node socket_server.js
TCP socket server is running on port: 6000
Client connected
Received: sub,95,42.4
Calculation Result: 52.6
Socket Error: read ECONNRESET
```

```
Terminal 2
$ node socket_client.js
Connected
Received: 52.6
```

Screenshot 2 – Baseline single client exchange (server and client terminals showing one request and one correct response).

This establishes that the basic protocol and arithmetic logic were functioning correctly before any load or modifications were introduced.

1.3 Baseline Repeated Calls (Provided Behaviour)

A second baseline screenshot shows repeated calls from the client using the original task behaviour (before adding any randomisation or new intervals). The client simply sends a sequence of operations at the default timing, and the server responds with correct results each time.

```

// socket_client.js
const client = net.createConnection(port, host, () => {
  console.log("Connected");
  setInterval(() => {
    client.write('sub,95,42,4');
  }, 1000); // Interval in milliseconds: (1000ms = 1 seconds)
});

client.on("data", (data) => {
  console.log("Received: " + data);
  // process.exit(0);
});

```

```

HPG3107@HP4364 /c/Users/HP/slt314_week1
$ node socket_server.js
TCP socket server is running on port: 8080
Client connected
Received: sub,95,42,4
Calculation Result: 52.6
Received: sub,95,42,4
Calculation Result: 52.6
Received: sub,95,42,4
Calculation Result: 52.6

```

```

HPG3107@HP4364 /c/Users/HP/slt314_week1
$ node socket_client.js
Connected
Received: 52.6
Received: 52.6
Received: 52.6

```

Screenshot 3 – Baseline repeated calls from the client using the provided code (multiple correct request–response cycles).

This serves as the reference point against which all later experiments are compared.

2. Client and Server Design

2.1 Server Logic (socket_server.js)

The server uses `net.createServer()` to accept incoming connections and attaches handlers for `data`, `end`, and `error` events.[1]

Core behaviour:

- On `data`, the server converts the buffer to a string and splits on commas.
- The first field is treated as the operator (`add`, `sub`, `mul`, `div`).
- The second and third fields are parsed as floating-point operands.

- A switch statement performs the requested operation:
 - add: `operand1 + operand2`
 - sub: `operand1 - operand2`
 - mul: `operand1 * operand2`
 - div: `operand1 / operand2` (with an explicit divide-by-zero check)
- Unknown operators fall into the default case, log an error, and produce NaN.
- The result is converted to a string and written back to the client.

This design is simple but effective for illustrating request parsing, error handling (unknown operator, divide by zero), and synchronous arithmetic in an asynchronous TCP server.

2.2 Client Logic (`socket_client.js`)

The client uses `net.createConnection(port, host, callback)` to connect to the server and then sends periodic requests using `setInterval().[1]`

In the final experimental version, the core behaviour is:

- Choose a random operator from `['add', 'sub', 'mul', 'div']`.
- Generate two random integers between 1 and 100.
- Format the request as a comma-separated string `operator,num1,num2`.
- Write the string to the server via the socket.
- Log both sent messages and received results to the terminal.

This design makes it easy to vary the interval, change the distribution of operations, and test the server under different loads.

3. Experiment A – Random Operators and Operands

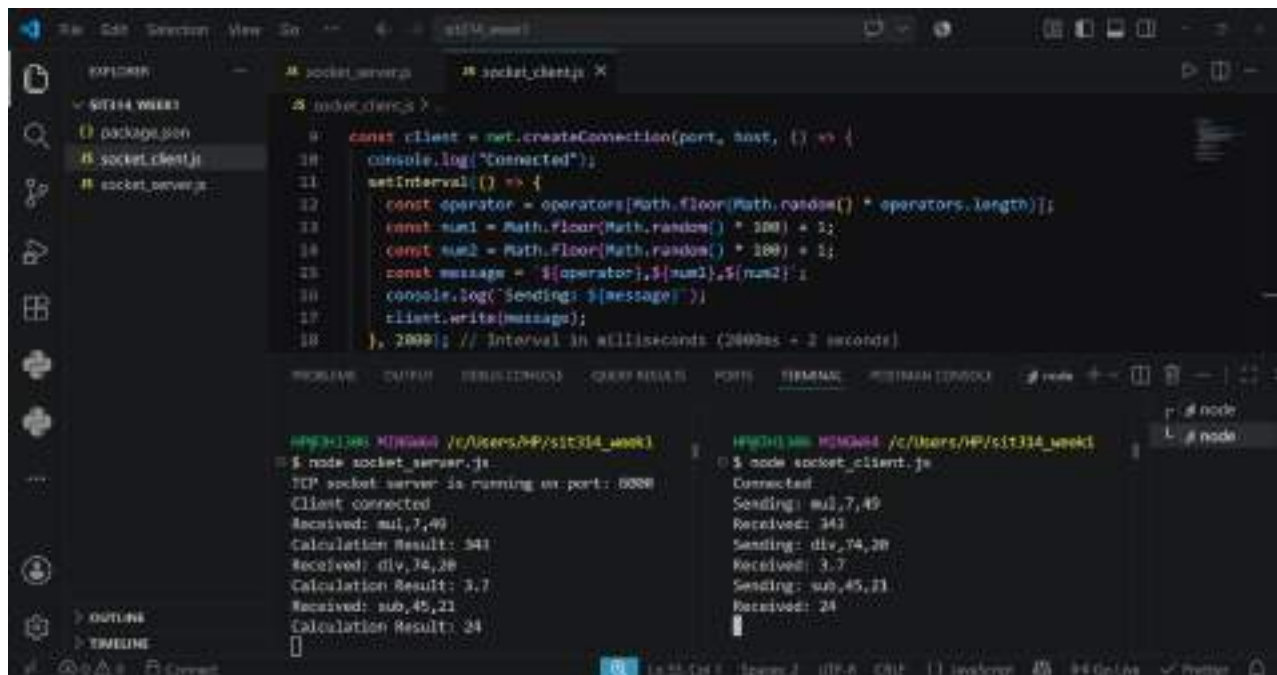
3.1 Modification

The first self-directed experiment was to modify the client to generate random operations and operands, rather than sending fixed test cases.

The following changes were made:

- Defined an `operators` array containing `add`, `sub`, `mul`, and `div`.
- Within `setInterval`, selected a random operator by index.
- Generated `num1` and `num2` as random integers from 1 to 100.
- Constructed the message string using template literals.

3.2 Evidence



```
socket_client.js
1  const client = net.createConnection(port, host, () => {
2    console.log("Connected");
3    setInterval(() => {
4      const operator = operators[Math.floor(Math.random() * operators.length)];
5      const num1 = Math.floor(Math.random() * 100) + 1;
6      const num2 = Math.floor(Math.random() * 100) + 1;
7      const message = `${operator},${num1},${num2}`;
8      console.log(`Sending: ${message}`);
9      client.write(message);
10     }, 2000); // Interval in milliseconds (2000ms = 2 seconds)
11   });
```

```
HP83138: MINGW64 /c/Users/HP/sit314_week1
$ node socket_server.js
TCP socket server is running on port: 5000
Client connected
Received: mul,7,49
Calculation Result: 343
Received: div,74,20
Calculation Result: 3.7
Received: sub,45,21
Calculation Result: 24

HP83138: MINGW64 /c/Users/HP/sit314_week1
$ node socket_client.js
Connected
Sending: mul,7,49
Received: 343
Sending: div,74,20
Received: 3.7
Sending: sub,45,21
Received: 24
```

Screenshot 4 – Random operator and operand generation (client code showing random selection and terminal showing mul/div/sub with varying values).

In this screenshot, the client logs messages such as:

- Sending: `mul,7,49` → server responds 343.
- Sending: `div,74,20` → server responds approximately 3.7.
- Sending: `sub,45,21` → server responds 24.

The mixture of operations and changing operands confirms that the server handles unpredictable input correctly and that the random generation logic is working as intended.

4. Experiment B – Interval Variation

4.1 From 2000ms to 50ms

The task explicitly suggested varying the interval to increase load. The initial interval was 2000ms (2 seconds), which was reduced to 50ms (20 requests per second) for stress testing.

This change multiplies the request rate by approximately 40 while keeping the message format and server logic unchanged.

4.2 Evidence and Observations

```

// socket_client.js
const net = require('net');
const port = 3000;
const host = 'localhost';

const client = net.createConnection(port, host, () => {
  console.log('Connected');
  setInterval(() => {
    const operator = operators[Math.floor(Math.random() * operators.length)];
    const num1 = Math.floor(Math.random() * 100) + 1;
    const num2 = Math.floor(Math.random() * 100) + 1;
    const message = `${operator},${num1},${num2}`;
    console.log('Sending: ', message);
    client.write(message);
  }, 50); // Interval reduced from 2000ms to 50ms for stress testing
});

// socket_server.js
const net = require('net');
const port = 3000;
const host = 'localhost';

const server = net.createServer((socket) => {
  socket.on('data', (data) => {
    const [operator, num1, num2] = data.split(',').map(Number);
    let result;
    switch (operator) {
      case '+':
        result = num1 + num2;
      case '-':
        result = num1 - num2;
      case '*':
        result = num1 * num2;
      case '/':
        result = num1 / num2;
    }
    console.log('Received: ', data);
    console.log('Calculation Result: ', result);
    socket.write(`${result}`);
  });
});

server.listen(port, host, () => {
  console.log('Server listening on port 3000');
});
  
```

Terminal Output:

```

Received: div,67,46
Calculation Result: 1.4565217291304348
Received: sub,74,79
Calculation Result: -5
Received: div,63,38
Calculation Result: 2.25
Received: sub,42,86
Calculation Result: -46
Received: mul,65,66
Calculation Result: 4420
Socket Error: read ECONNRESET
  
```

Screenshot 5 – Client and server at 50ms interval (rapid sequence of correct calculations without corruption).

Despite the much higher rate, the server:

- Correctly parsed all incoming messages.
- Produced accurate arithmetic results for each operation.
- Did not log any parsing errors or NaN results.

This demonstrates that, at moderate stress levels, Node.js's event-driven I/O model and the simple server logic are sufficient to handle sequential high-frequency requests without data loss.[2]

5. Experiment C – Multiple Simultaneous Clients

5.1 Setup

To test concurrency, three separate terminal sessions were opened and node socket_client.js was run in each, all connecting to the same server process and port.

5.2 Evidence

The screenshot displays four terminal windows. The top window is the server terminal, showing the execution of 'node socket_server.js'. It logs 'TCP socket server is running on port: 8080' and then receives three client connections. The received data is interleaved: 'sub,31,2', 'mul,92,72', 'sub,86,17', 'mul,12,44', 'sub,72,18', and 'add,30,28'. The bottom three windows are client terminals, each running 'node socket_client.js'. Each client shows its own 'Connected' message and a sequence of operations and results: Client 1 (sub,31,2 → 29, mul,12,44 → 528), Client 2 (mul,92,72 → 6624, sub,80,17 → 63), and Client 3 (mul,5,69 → 345, sub,77,11 → 66, add,94,55 → 149, mul,50,38 → 1900).

Screenshot 6 – Three simultaneous client terminals plus one server terminal.

The server terminal shows:

- Client connected logged three times.
- Interleaved Received: ... lines corresponding to different clients.

Each client terminal shows:

- Its own Connected message.
- Independent random sequences of operations and results, e.g.:
 - Client 1: sub,31,2 → 29, mul,12,44 → 528.
 - Client 2: mul,92,72 → 6624, sub,80,17 → 63.

- Client 3: mul,5,69 → 345, sub,77,11 → 66.

The server responds correctly to each client without mixing their results, showing that `net.createServer()` can manage multiple concurrent TCP connections on a single thread by multiplexing events.[1]

6. Experiment D – Cross-Network Connection

6.1 Identifying Network Interface

Because the task encouraged connecting "between other computers," the next step was to move beyond `localhost` and use the machine's Wi-Fi interface.

An `ipconfig` command was run in Windows Command Prompt, producing output showing several adapters and their IP addresses.[3]

Key lines:

- **Wi-Fi adapter** (`wifi-m.deakin.edu.au`):
 -
 - IPv4 Address: `10.141.14.108`.
 - Default Gateway: `10.141.0.1`.

This identified `10.141.14.108` as the correct host address for the server.

```
C:\Users\HP>ipconfig

Windows IP Configuration

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . : wifi-m.deakin.edu.au
    Link-local IPv6 Address . . . . . : fe80::223a:32c:101f:f025%5
    IPv4 Address. . . . . : 10.141.14.108
    Subnet Mask . . . . . : 255.255.192.0
    Default Gateway . . . . . : 10.141.0.1
```

Screenshot 7 – `ipconfig` output highlighting Wi-Fi IPv4 address (`10.141.14.108`).

6.2 Reconfiguring Client and Server

The following changes were made for the network experiment:

- In `socket_client.js`, host was set to "10.141.14.108" instead of "127.0.0.1".
- The server continued using `server.listen(port, callback)` without specifying a host, which in Node.js defaults to binding to all interfaces (effectively 0.0.0.0).[1]

The client and server were then run again.

6.3 Evidence and Outcome

The screenshot shows the Visual Studio Code editor with two JavaScript files open: `socket_client.js` and `socket_server.js`. The `socket_client.js` file has the following code:

```
//socket_client.js
1 const net = require('net');
2
3
4 const host = "10.141.14.108";
5 const port = 6000;
6
7 const operators = ["add", "sub", "mul", "div"];
8
9 const client = net.createConnection(port, host);
```

The `socket_server.js` file has the following code:

```
1 const net = require('net');
2
3
4 const port = 6000;
5
6 server.listen(port, () => {
7   console.log('TCP socket server is running');
8 });
9
10 server.on('error', (error) => {
11   console.log('Server Error: ${error.message}');
12 });
13
14 server.on('data', (data) => {
15   const str = data.toString().trim();
16   const [operator, ...operands] = str.split(' ');
17   if (operators.includes(operator)) {
18     let result = 0;
19     for (let i = 0; i < operands.length; i++) {
20       result = eval(`${result}${operator}${operands[i]}`);
21     }
22     console.log(`Calculation Result: ${result}`);
23   } else {
24     console.log('Invalid input');
25   }
26 });
```

Below the editor, two terminal windows are open. The left terminal shows the output of running `node socket_server.js`:

```
HP@HILM: /c/Users/HP/sit314_week1
$ node socket_server.js
TCP socket server is running on port: 6000
Client connected
Received: mul,30,54
Calculation Result: 1620
Received: sub,34,6
Calculation Result: 28
Received: add,2,73
Calculation Result: 75
Received: sub,20,4
Calculation Result: 16
Received: div,99,63
```

The right terminal shows the output of running `node socket_client.js`:

```
HP@HILM: /c/Users/HP/sit314_week1
$ node socket_client.js
Connected
Sending: mul,30,54
Received: 1620
Sending: sub,34,6
Received: 28
Sending: add,2,73
Received: 75
Sending: sub,20,4
Received: 16
Sending: div,99,63
Received: 1.5734285714285714
```

Screenshot 8 – Server and client communications using the Wi-Fi IP (10.141.14.108).

The terminals show correct calculations for operations like:

- `mul,30,54` → 1620.
- `sub,34,6` → 28.
- `add,2,73` → 75.
- `div,99,63` → approximately 1.5714.

This confirms that the application works over the actual network stack, not just over the loopback interface. The same logic and protocol succeed when traversing the Wi-Fi interface, which is closer to a real deployment scenario.[4]

7. Experiment E – Malformed Input and TCP Stream Effects

7.1 Motivation and Changes

To explore how the server behaves under invalid or malformed requests, additional messages were injected from the client:

- "add,5" – missing second operand.
- "mul,abc,5" – non-numeric operand.
- "pow,2,3" – unknown operator.
- "div,10,0" – divide-by-zero case.

These were sent rapidly via multiple `setTimeout` calls in addition to the normal random traffic.

7.2 Observed Behaviour

On the server side, the output shows a mix of valid calculations and repeated NaN results:

- Valid examples: `mul,87,54` → 4698, `div,47,96` → approximately 0.4896.
- Invalid examples:
 - `add,5` → Calculation Result: NaN (missing operand).
 - `mul,abc,5` → Calculation Result: NaN (parseFloat fails).
 - `pow,2,3` → Unknown operator: pow, Calculation Result: NaN.
 - `div,10,0` → Division by zero attempted, Calculation Result: NaN.

More importantly, at high message rates, several lines show multiple messages merged into one string, such as:

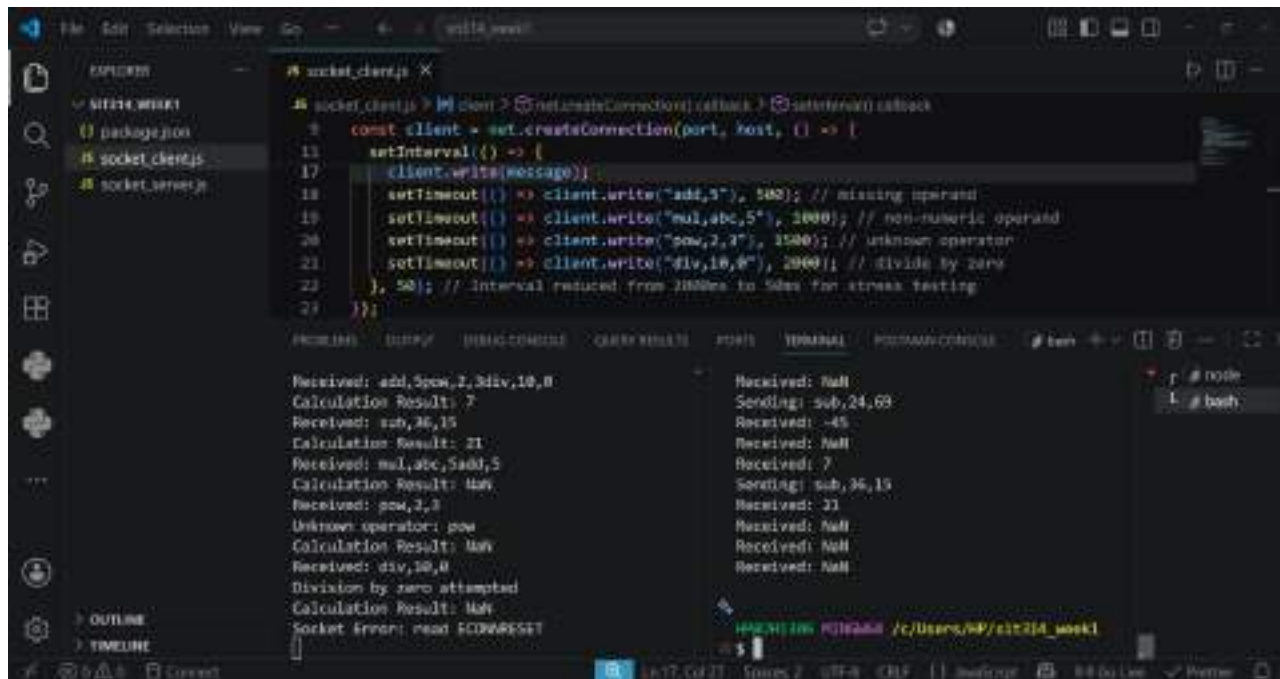
- Received: `add,5mul,abc,5`.
- Received: `mul,abc,5pow,2,3div,10,0`.
- Received: `sub,72,62mul,29,75div,41,18`.

These concatenated lines are a direct symptom of TCP's **stream** nature: TCP does not preserve message boundaries, so multiple writes from the client can arrive as a single burst of bytes and be delivered in one data event. [5] [6]

The server's parsing logic assumes each data event contains exactly one comma-separated message. Under rapid, malformed traffic, that assumption breaks, leading to incorrect parsing and NaN results even when parts of the merged string are valid.

Eventually, the server logs:

- Socket Error: read ECONNRESET.



Screenshot 9 – Server and client terminals during malformed input experiment (merged messages, NaN results, and ECONNRESET).

This experiment successfully "broke" the application by exposing the lack of message framing in the TCP protocol and the fragility of the simple parsing logic.[7]

8. Experiment F – Extreme Stress Test (1ms Interval)

8.1 Configuration

For the final experiment, the client's interval was reduced from 50ms to 1ms:

```
setInterval(() => {
  const operator = operators[Math.floor(Math.random() * operators.length)];
  const num1 = Math.floor(Math.random() * 100) + 1;
  const num2 = Math.floor(Math.random() * 100) + 1;
```

```
const message = `${operator},${num1},${num2}`;
client.write(message);
}, 1);
```

This pushes the server to handle approximately 1000 requests per second per client. In some runs, several seconds of traffic were generated; in others, the connection failed more quickly.

8.2 Observed Behaviour

The long server log shows hundreds of correct calculations under this extreme load, including many diverse operations and operands. Examples include:

- mul,90,36 → 3240.
- div,84,4 → 21.
- sub,75,9 → 66.
- div,10,72 → approximately 0.1389.

Intermittent concatenation issues reappear due to the high write rate (e.g. Received: div,49,67add,14,34, mul,91,64add,57,67), and the server continues to produce NaN for invalid or badly merged inputs.[8]

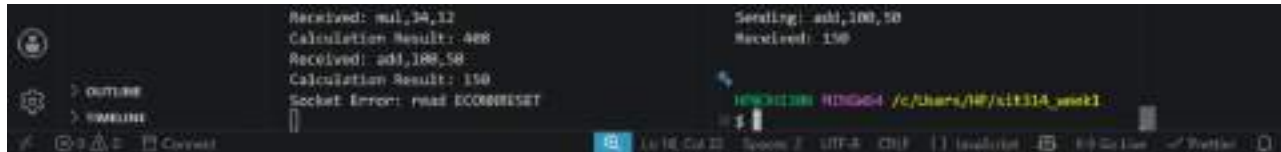
Eventually, after sustaining this heavy load for a significant time, the server logs:

- Socket Error: read ECONNRESET.

On the client side, output stops after a final set of Sending: and Received: lines, indicating that the connection was forcibly reset by the peer and the client process exited or stalled.

```
socket_client.js
1 // client > net.createConnection(port, host, () => {
2   // setInterval() => {
3     const operator = operators[Math.floor(Math.random() * operators.length)];
4     const num1 = Math.floor(Math.random() * 100) + 1;
5     const num2 = Math.floor(Math.random() * 100) + 1;
6     const message = `${operator},${num1},${num2}`;
7     console.log('Sending: ' + message);
8     client.write(message);
9   }, 1); // extreme stress test
10 }
```

```
Received: add,40,28
Calculation Result: 72
Received: add,6,40
Calculation Result: 32
Received: mul,56,87
Calculation Result: 4872
Sending: add,5,40
Received: 52
Sending: mul,56,87
Received: 4872
Sending: mul,34,12
Received: 408
```



Screenshot 10 – Extreme stress test at 1ms interval (long sequence of valid results, intermittent merges, ending in ECONNRESET).

8.3 Interpretation

This experiment demonstrates that:

- The server is surprisingly robust, handling many hundreds of requests per second for an extended period.
- At very high write rates, message merging and parsing errors become increasingly frequent because TCP delivers data as a stream, not discrete packets.[5]
- Connection resets (ECONNRESET) eventually occur, representing a practical breaking point where the current design can no longer maintain a stable session.

A production-grade design would typically:

- Use explicit message framing (e.g. appending "\n" and splitting on newlines or using length-prefixed messages).[9]
- Maintain a buffer and parse complete messages incrementally from the stream.

These mitigations are beyond the scope of this Week-1 task but are worth mentioning as future improvements.

9. Summary of Experiments

The table below summarises each experiment, the change made, and its outcome.

Experiment	Change / Setup	Key Observations	Evidence
Baseline single call	Original client/server, one request	Correct arithmetic, working TCP connection	Screenshot 2

Baseline repeated calls	Original repeated client behaviour	Sustained correct results at default interval	Screenshot 3
Random operations & operands	Client chooses random operator and numbers each interval	Server handles unpredictable input (mul/div/sub with varying values)	Screenshot 4
Interval 2000ms → 50ms	Higher frequency (≈20 requests/sec)	No corruption, accurate results, stable connection	Screenshot 5
3 simultaneous clients	Three client processes connected concurrently	Three "Client connected" logs, interleaved Received lines, correct per-client results	Screenshot 6
Cross-network via Wi-Fi IP	Host set to 10.141.14.108, server bound to all interfaces	Correct results over actual network stack, not just localhost	Screenshot 7 & 8
Malformed input + merges	Injected add,5, mul,abc,5, pow,2,3, div,10,0 at high rate	NaN results, unknown-operator logs, merged messages due to TCP stream, eventual ECONNRESET	Screenshot 9
Extreme stress (1ms)	Interval reduced to 1ms, long run	Hundreds of correct results, frequent merges, final ECONNRESET, clear breaking point	Screenshot 10

10. Real-World Relevance

These experiments mirror challenges faced in real networking and IoT systems:

- **Load and concurrency:** Servers must handle many clients and high request rates without corrupting data or crashing. The multi-client and interval tests show how event-driven architectures can scale until physical or protocol limits are reached.[4]
- **Message framing:** Real protocols (HTTP, WebSocket, MQTT) include framing mechanisms or length headers specifically to avoid TCP stream boundary issues seen here.[10]
- **Error handling:** Robust systems must handle malformed input gracefully, including unknown operations and divide-by-zero cases, without bringing down the server.
- **Network environment:** The cross-network experiment highlights that university or enterprise networks often impose constraints (firewalls, client isolation) that affect peer-to-peer communication, which must be considered when deploying IoT or distributed applications.[11]

By intentionally "breaking" the simple client-server application in controlled ways, this Week-1 technical task provides a practical foundation for later units that introduce more complex protocols, message formats, and resilience techniques.

References

1. [Net | Node.js v26.5.0 Documentation](#) - The node:net module provides an asynchronous network API for creating stream-based TCP or IPC server...
2. [loadtest/doc/tcp-sockets.md at main · alexfernandez/loadtest](#) - Runs a load test on the selected URL. Fast and easy to use. Can be integrated in your own workflow u...
3. [ipconfig](#) - Reference article for the ipconfig command, which displays all current TCP/IP network configuration ...
4. [Socket performance testing guidelines](#) - Before you can test the performance of TCP/IP socket-based applications, set up your test environmen...
5. [Is code that relies on packet boundaries for a TCP stream inherently ...](#) - From what I understand, TCP is a stream, and shouldn't rely on packets, or boundaries of any kind. A...
6. [Chapter 1. Introduction - Shichao's Notes](#) - Protocols that do not preserve message boundaries (e.g., streaming protocols like TCP) ignore this i...
7. [Split messages over tcp](#) - Im having a little trouble in a uni project. Im sending a tcp message from a C++ client to a C# serv...
8. [Solution for Message Merging into One Due to High Frequency Messaging in Node.js Socket](#)
9. [How to keep Node.js from splitting socket messages into smaller chunks](#) - I've got a chat program which pushes JSON data from Apache/PHP to Node.js, via a TCP socket: // Node...
10. [TCP Server in Zig - Part 2 - Message Boundaries](#) - TCP is just a continuous stream of bytes, it has no concept of message boundaries. It's possible, an...
11. [Network Stress Tests: How to Stress Test a Network Safely](#) - Knowing how to stress test a network helps optimize your system and provides insight into changes ne...

6/17/2004 11:04:23 AM

more and more people.

Strongly Recommended

© 2004 Blackwell Publishing Ltd *Journal of Internal Medicine* 255: 101–108

© 2001 Blackwell Science Ltd

© 2005 Blackwell Publishing Ltd *Journal of Internal Medicine* 258: 103–110

© 2004 Blackwell Publishing Ltd

intensity across the months.

SIT729 - Week 2 Seminar

- Week 2 Session
- How was your weekend?
- Class Discussion
- TI
- Class Discussion
- Class Discussion
- Class Discussion
- Class Discussion
- Class Discussion
- Class Discussion

How was your weekend?



- 3 Great
- 2 Relaxed
- 7 Working!
- 5 Tiring
- 3 We had a weekend?

John Gong (my)

Active

Week 2 Friday 17 July - Mentimeter

meritnation.com/app/promotion/slwtoqe1d5o42f6w4f96p5ggpu1d/view?utm_source=google&utm_medium=button&utm_campaign=efwibmef4gpp

Week 2 Friday 17 July

View only

IoT20 - Week 2 Session

How well you understand?

Class Discussion

IoT20 - Week 2 Session

How well you understand?

IoT20 - Week 2 Session

How well you understand?

IoT20 - Week 2 Session

How well you understand?

IoT20 - Week 2 Session

How well you understand?

IoT20 - Week 2 Session

How well you understand?

Class Discussion

1. IoT Nodes

2. IoT Communication

3. Cloud Computing for IoT

12:44

17-07-2020

Week 2 Friday 17 July - Mentimeter

mentimeter.com/app/presentation/3kwtoge1d5ca2f8e4f896p5ggpu1d/view?utm_source=google&utm_medium=button&utm_campaign=gskq5a5f3j

Week 2 Friday 17 July

View only

1
2
3
4
5
6
7
8

Week 2 Session
How well you understand
Class Discussion
**TI
Devices and Platform slides**
1 of 2 Slides
Where would you like to go
What technology concepts
Identify some key trends

DEAR

Devices and Platform slides - 2.1 (1-9)

Search

Taskbar icons: File Explorer, Microsoft Edge, Teams, Word, PowerPoint, OneDrive, etc.

System tray: Date/Time (17-07-2020 12:44), Network, Volume, Battery (91%)

Week 2 Friday 17 July - Mentimeter

meritnation.com/app/presentation/3wrtoge1d15o42fwh496p5ggpu1d/view?utm_source=google&utm_medium=button&utm_campaign=1d5w4r1d15o42fwh496p5ggpu1d

Week 2 Friday 17 July

View only

IoT - Week 2 Session

How well you understand?

Class Discussion

IoT Nodes

Where is your data going?

What happens when you click?

Identifying some IoT nodes in...

1. IoT Nodes

12:44

17-07-2020

Week 2 Friday 17 July

View only

What capabilities does an IoT node need?

A word cloud centered around the question 'What capabilities does an IoT node need?'. The background is a light blue sky with a city skyline. The words are in various colors and sizes, representing different IoT capabilities. The most prominent words are 'internet connection', 'connection', 'flexible', 'connectivity', 'sensing', 'power', 'low power', 'wi-fi', and 'bluetooth'. Other words include 'scalability', 'connect to sensors', 'internet protocols', 'connect to hardware', 'wide range', 'real-time processing', 'electricity', 'share data', 'data storage', 'data capture', 'router devices', 'communicate with others', 'bridge hardware and cloud', 'sensing and actuating', 'security', 'data transmission', 'connect to network', 'power management', 'longevity', 'data collection', 'able to capture data', 'communication interface', 'compression', 'connect to cloud', and 'data storage'.

IoT201 Week 2 Session

How will you connect it?

Cloud Resources

IoT201 Week 2 Session

What capabilities does an IoT node need?

What hardware connects?

Security comes for IoT nodes too

12:44

17-07-2020

Week 2 Friday 17 July

View only

What hardware components does an IoT node need?



A word cloud titled "What hardware components does an IoT node need?". The words are arranged in a cloud shape against a background of a modern building with glass windows. The most prominent words in the center are "gateways" (large blue), "cpu" (large yellow), "wifi" (large pink), and "power supply" (large purple). Other visible words include "sensors" (green), "actuators" (green), "memory" (blue), "microcontroller" (red), "network adapter" (small blue), "radio" (small blue), "gateway" (small blue), "microcontrollers" (small green), "is power supply" (small yellow), "battery" (small green), "router" (small green), "gsm" (small green), "microprocessor" (small red), "module" (small green), "port" (small blue), "ethernet" (small blue), "base" (small blue), "mcu" (small blue), "internet adapter" (small red), "compact lightweight design" (small red), "arduino" (small yellow), and "communication" (small yellow). The words vary in size and color, with blue and green being the most frequent colors.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

Week 2 Friday 17 July - Mentimeter

mentimeter.com/app/presentation/3lw7oge1d05c42f64d996p5jg9u1d/view?utm_source=google&utm_medium=button&utm_campaign=on91aeg7any

Week 2 Friday 17 July

View only

Play with your audience!

Open Discussion

TI

2 Left Slides

When you finish this slide

When navigating amongst slides

Display a new full screen slide

TI

Device Platforms slides - 2.3

12:45

17-07-2020

Week 2 Friday 17 July

View only

2

Open Document

3

T1

Business and Technology

4

IoT Models

5

After registration class 1

6

After registration workshop

7

After registration workshop

8

Identity Issues and Security 1

9

Security Workshop class 1

10

IoT Communications

meritnation.com/app/promotion/3/wtoge1d5c42f64f96p5g9u1d/view?utm_source=google&utm_medium=button&utm_campaign=Q&adtype6

2. IoT Communications

12:45

17-07-2020

Week 2 Friday 17 July - Mento

mentimeter.com/app/presentation/3lw7oge1d5c42f64f96p5g9e1d4/view?utm_source=google&utm_medium=button&question=27qaw5965c

Week 2 Friday 17 July View only

What IoT communications technologies exist?

cloud ethernet wifi pixel drop wireless fidelity http switch tcp ip lorah open thread nano nfc protocols like coap zigbee airdrop cellular bluetooth low energy router satellite dish zwave tcp

1 IoT Basics
2 What is IoT?
3 What is IoT communication?
4 IoT communication protocols
5 Security in IoT
6 IIOT
7 IIOT communication
8 IIOT communication
9 IIOT communication
10 IIOT communication

12:45 17-07-2020

[illegible]

Week 2 Friday 17 July - Mentimeter

mentimeter.com/app/presentation/3lw7oge1d5c42f64d896p5jggu1d4/view?utm_source=google&utm_medium=button&question=1y56df1ockd7

Week 2 Friday 17 July

View only

7

What features are essential?

8

Identify some IoT features...

9

Describe Platform capabilities...

10

IoT is a...

11

Identifying cloud IoT use...

12

What IoT communication...

13

Identify some IoT features...

14

Prototyping scalable IoT

1234567891011121314

Prototyping Scalable IoT devices - 2.2 1-9

12:45

17-07-2020

Week 2 Friday 17 July

View only

23

1

Security issues for IoT...

2

Future IoT devices and...

3

IoT Communications

4

Introducing cloud IoT arch...

5

Cloud IoT Communications

6

Security issues for IoT...

7

Future IoT devices and...

8

IoT Communications for IoT

3. Cloud Computing for IoT

12:45

17-07-2020





Jahan Garg | Student ID: 226100103

SIT314/SIT729 – Week 2 Group Activity

Technologies for IoT Applications

Overview

This activity builds on the concepts covered in Module 1, Topics 1.7 to 1.9 (Devices and Platforms for Scalable IoT, Prototyping Scalable IoT Devices, and Device Platforms), which introduced end-user devices, network devices, cloud servers, and the range of prototyping and commercial platforms available for building IoT solutions [1][2][3]. Now that a reasonable understanding of the options for IoT nodes and communications has been established, this task explores which technologies would be most appropriate for six distinct application scenarios, ranging from a domestic smart lighting system to autonomous robotic maintenance on Mars.

Note on Individual Submission: This activity was discussed collaboratively during the SIT314 seminar on Friday, 17 July 2026, with fellow group members Melissa Woo and Madhur Gupta, where different technology options and design trade-offs for each scenario were debated as a group. However, in line with OnTrack's individual submission requirement for module evidence, the written analysis, technology justifications, and referencing presented below are my own independent work, reflecting my personal understanding of the concepts.

Tasks

Consider the following applications and answer the questions about the most appropriate technologies to use.

1. A smart home lighting system.	
What properties does your IoT node need?	The node must support real-time control of dimming, colour, and scene presets, operate at low power on a continuous basis, and maintain secure device identity together with local storage for schedules and fail-safe defaults if connectivity drops [4].
What microcontroller would you use?	An ESP32-based board is appropriate, since it combines a dual-core processor with integrated Wi-Fi and Bluetooth Low Energy, enough memory for MQTT clients and encryption, and a wide industrial operating temperature range of -40C to 105C at low hardware cost [5].

What communications standard would you use?	A low-power mesh protocol such as Zigbee or Thread would connect bulbs and switches to a central hub, since mesh topologies are designed for dense device networks and can route around individual node failures; the hub then uses Wi-Fi and MQTT to reach the cloud [2].
What edge computing infrastructure would you use?	A local hub, similar to a Raspberry Pi-based gateway or a commercial hub such as SmartThings, would run automation rules (motion-triggered lighting, scheduled scenes) so that basic lighting control continues even without internet access [6][2].
Would you use cloud computing?	Yes, but in a supporting role. Cloud services would provide remote access via a mobile application, long-term energy analytics, and integration with third-party ecosystems, while safety-critical and time-sensitive control logic remains on the local hub [1][2].

2. A forest fire monitoring system.

What properties does your IoT node need?	The node needs to withstand harsh outdoor conditions, operate for extended periods on battery or solar power, capture temperature, humidity, and smoke measurements, and timestamp and geotag each reading for later correlation [7].
What microcontroller would you use?	A low-power ARM Cortex-M microcontroller on a ruggedised LoRa-enabled board is suitable, as this class of chip supports deep sleep modes with periodic wake-ups for sensing and transmission, extending field deployment life significantly [8].
What communications standard would you use?	LoRaWAN is appropriate given the large, sparsely connected forest area, since it is a low-power wide-area network protocol purpose-built to connect battery-operated devices across kilometre-scale distances with small periodic payloads [9][10].
What edge computing infrastructure would you use?	Regional LoRaWAN gateways would aggregate data from many field nodes and run simple anomaly detection (e.g., rapid temperature or smoke spikes) locally, enabling faster alerts than waiting for centralised cloud processing [9].
Would you use cloud computing?	Yes, cloud computing would store historical datasets, run predictive fire-risk models, and provide dashboards for emergency services, while the edge gateways handle time-critical alerting independently of cloud availability [7][9].

3. A smart air-conditioning system for a large building.

What properties does your IoT node need?	Nodes need to monitor zone temperature, humidity, occupancy, and air quality, interact reliably with damper and fan actuators, and support tightly closed feedback loops together with safety interlocks integrated into the building management system [11].
What microcontroller would you use?	An industrial-grade controller, such as a Raspberry Pi Compute Module or a dedicated building automation PLC, is appropriate, as these platforms can interface with fieldbus protocols and run more complex control logic than a simple microcontroller [11].

What communications standard would you use?	Wired Ethernet using BACnet/IP or Modbus TCP is recommended for core HVAC control loops because these standards were purpose-built for building automation and provide reliable, low-latency, interoperable communication between controllers and sensors [12][13].
What edge computing infrastructure would you use?	A building-level server hosting the Building Management System would aggregate zone data, run optimisation algorithms, and provide local dashboards to facility managers, ensuring HVAC operation continues even if cloud connectivity is interrupted [12].
Would you use cloud computing?	Yes, cloud computing would support fleet-level analytics across multiple buildings, long-term energy benchmarking, and remote maintenance scheduling, while core control logic remains on-premises for resilience [11][12].

4. A river monitoring system for the remote Australian outback.

What properties does your IoT node need?	Nodes need to withstand extreme temperature swings, flooding, and wildlife interference, measure water level, flow rate, and turbidity, and operate for long unattended periods on solar power with remote diagnostic capability [7].
What microcontroller would you use?	A ruggedised, waterproof board built around an ARM Cortex-M microcontroller with integrated sensor interfaces and power management would allow local preprocessing, such as averaging readings, to reduce transmission volume [8].
What communications standard would you use?	Given the remoteness of the Australian outback, LoRaWAN or narrowband satellite IoT connectivity is most suitable, as LoRaWAN is explicitly designed for long-range, low-power connectivity where cellular infrastructure is absent [9][10].
What edge computing infrastructure would you use?	Small solar-powered gateway stations positioned at intervals along the river would buffer data during backhaul outages and run basic threshold checks to flag sudden changes that may indicate flooding or contamination [9].
Would you use cloud computing?	Yes, cloud computing would consolidate data from multiple monitoring sites and support dashboards for water authorities, but the design must anticipate intermittent connectivity and rely on robust local buffering [7][9].

5. A driver-less taxi system for a smart city.

What properties does your IoT node need?	The on-board computing unit must process high-throughput sensor data from cameras, lidar, radar, and inertial measurement units in real time, support low-latency safety-critical decisions, and maintain continuous health monitoring of hardware and software [14].
What microcontroller would you use?	A single microcontroller is insufficient; an automotive-grade, GPU-accelerated compute platform is required to run perception and planning algorithms, with secondary microcontrollers handling non-critical subsystems such as door locks [14].

What communications standard would you use?	In-vehicle communication would use Automotive Ethernet and CAN buses, while vehicle-to-infrastructure and vehicle-to-vehicle coordination would rely on Cellular-V2X over 5G networks to exchange safety-critical traffic information [15].
What edge computing infrastructure would you use?	Roadside edge servers would aggregate data from multiple vehicles, distribute local map and traffic updates, and support low-latency coordination at intersections without depending on distant cloud data centres for time-critical decisions [15].
Would you use cloud computing?	Yes, for fleet management, long-term logging, model training, and over-the-air updates, but active driving decisions must remain at the edge since safety cannot depend on a live round trip to the cloud [14][15].

6. A system for automatic robotic maintenance on Mars.	
What properties does your IoT node need?	Nodes must tolerate extreme cold, dust storms, and high radiation levels, monitor robot health and power status, and support autonomous decision-making given communication delays of several minutes to Earth [16].
What microcontroller would you use?	A radiation-hardened processor such as the RAD750, used in multiple NASA missions including the Curiosity and Perseverance rovers, is necessary; it withstands up to 1,000,000 rads of radiation and operates between -55C and 125C [16][17].
What communications standard would you use?	Local robot-to-robot and robot-to-base communication would use UHF radio, while communication to Earth would depend on deep-space relay via orbiters, using store-and-forward protocols designed to tolerate high latency and disruption [17].
What edge computing infrastructure would you use?	A Mars-based control station would function as the primary computing tier, coordinating multiple robots and performing as much planning and analysis as possible locally, since round-trip communication with Earth is impractical for real-time control [16].
Would you use cloud computing?	Terrestrial cloud computing would still analyse returned mission data and refine algorithms for future updates, but because of communication delays, the robotic system must operate independently for extended periods using local infrastructure as the primary decision layer [16][17].

Reflection

Working through these six scenarios reinforced that there is no single best IoT technology stack; rather, the choice of microcontroller, communication standard, edge infrastructure, and reliance on cloud computing must be driven by the specific constraints of the deployment environment, including power availability, latency tolerance, physical hardening requirements, and the criticality of the application. Comparing a domestic lighting system with a Mars maintenance robot highlighted how the balance between edge and cloud processing shifts dramatically as communication delay and criticality increase, echoing the module discussion of the N-tier fog and edge deployment architecture [3]. Discussing these trade-offs with Melissa Woo and Madhur Gupta during the seminar was valuable in testing my initial assumptions, particularly around whether cloud computing should be treated as a

default choice; the group discussion helped clarify that cloud computing is best treated as a complement to, rather than a replacement for, local and edge processing in scalable IoT design.

References

- [1] Deakin University, "1.1 What is Scalability?," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [2] Deakin University, "1.7 Devices and Platforms for Scalable IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [3] Deakin University, "1.3 A Software Engineering Approach for Scalable IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [4] Deakin University, "1.6 Week 1 Technical Task," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [5] Espressif Systems, "ESP32 Wi-Fi and Bluetooth SoC," Espressif Systems, 2026. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>
- [6] Deakin University, "1.8 Prototyping Scalable IoT Devices," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [7] Deakin University, "1.2 Scalability in IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [8] Deakin University, "1.9 Device Platforms," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [9] LoRa Alliance, "What is LoRaWAN Specification," LoRa Alliance, 2024. [Online]. Available: <https://loro-alliance.org/about-lorawan-old/>
- [10] LoRa Alliance, "LoRaWAN Specification v1.1," LoRa Alliance, 2023. [Online]. Available: <https://resources.lora-alliance.org/technical-specifications/lorawan-specification-v1-1>
- [11] ASHRAE, "BACnet, the ASHRAE Building Automation and Control Networking Protocol," ASHRAE, 2020. [Online]. Available: <https://www.ashrae.org/technical-resources/bookstore/bacnet>
- [12] EMQ Technologies, "BACnet vs Modbus: Technical Differences and Selection Guide," EMQX, 2026. [Online]. Available: <https://www.emqx.com/en/blog/bacnet-vs-modbus>
- [13] Deakin University, "1.5 Week 1 Group Activity," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [14] Deakin University, "1.4 Week 1 Class Discussion," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [15] Deakin University, "1.10 Week 2 Class Discussion," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1: Scalability, CloudDeakin, 2026.
- [16] BAE Systems, "RAD750: Radiation-Hardened PowerPC Microprocessor," BAE Systems, 2025. [Online]. Available: <https://www.baesystems.com/en/dam/jcr:c297f318-c618-4015-9399-adf76ac5cf55/16-f78-RAD750-Component-datasheet-2025-web.2025-06-25>
- [17] Wikipedia, "RAD750," Wikimedia Foundation, 2025. [Online]. Available: <https://en.wikipedia.org/wiki/RAD750>